Instituto de Formación Técnica Superior N° 18

Tecnicatura en Ciencias de Datos e Inteligencia Artificial

Trabajo Integrador

Informe

Materia: Seminario de Actualización

2do Cuatrimestre 2026

Docente: Cynthia Villagra

Alumno: Angel Sebastián Medina

Aplicación: Traductor Inglés - Español

Resolución del Trabajo Práctico Integrador

**A. El problema**

1. ¿Qué problema querés resolver y para quién? ¿Por qué un modelo de IA y no unas reglas escritas a mano (un **if** con palabras clave)?

La idea de la aplicación surgió de una situación cotidiana en la cual un comerciante amigo, que posee una tienda de mascotas, me solicita apoyo para promocionar su negocio en la aplicación Google Maps mediante “estrellas” y una reseña. De allí me comenta que se ha encontrado con varias reseñas en inglés, debido a que en el barrio ha aumentado la cantidad de clientes extranjeros y para él resulta un problema ya que no sabe dicho idioma. Es por eso que se me ocurre crear una aplicación de traducción, en la cual el usuario pegue un comentario y pueda saber que significa y a su vez, redactar una respuesta que pueda ser traducida del español al inglés y además responder, aprovechando la funcionalidad del “Portapapeles” del celular de pegar un texto desde él, a las reseñas de Google Maps.

La elección de un modelo de IA por sobre un código con reglas a “mano” posee varias ventajas, entre las cuales puedo citar:

1) Un modelo de Machine Learning analizan la frase completa para poder comprender el contexto de ella y ofrecer traducciones similares a como hablaría un humana, mientras que el código tradicional traduce palabra por palabra sin estructura gramatical fija, imposibilitando la interpretación de dobles sentidos, modismos o metáforas.
2) La escalabilidad de la aplicación se ve facilitada enormemente. Al querer agregar nuevos idiomas o mejorar el elegido, se reentrena el modelo con más datos o se busca otro que reúna las características buscadas, mientras que en el código tradicional hay que agregar miles de nuevas líneas de código.
3) El mantenimiento de la aplicación es continuo. El feedback de los usuarios ayuda a corregir traducciones que sirven para reentranar el modelo, el cual mejora con el tiempo. En el código tradicional es más engorroso buscar un error puntual para ser corregido de manera manual.
4) El tiempo de desarrollo disminuye de forma abismal, ya que se aprovecha la existencia de modelos preentrenados (como en éste caso), dejando solamente a los desarrolladores la construcción de la interfaz y el deploy de la aplicación. Por otro lado, la construcción desde cero de un motor de traducción llevaría años de desarrollo, sin contar con la colaboración de otros profesionales.

**B. El modelo**

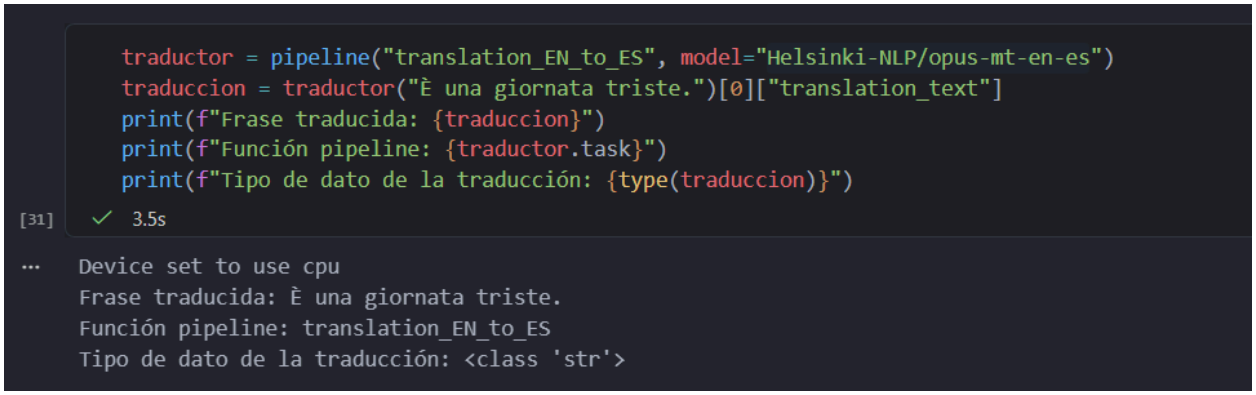
2. ¿Qué modelos consideraste y por qué te quedaste con uno? Citá lo que leíste en su **model card**: tarea, etiquetas, idiomas, datos de entrenamiento, tamaño de los pesos.

El par de modelos seleccionados para la aplicación (*Helsinki-NLP/opus-mt-es-en* y *Helsinki-NLP/opus-mt-en-es*) son modelos de traducción creados por la Universidad de Helsinki. Construidos en arquitectura Transformer, traducen del español al inglés y viceversa. Fueron entrenados con la colección OPUS (Open Parallel Corpus), una de las más grandes colecciones de corpus paralelos (textos alineados en dos idiomas) para el procesamiento de lenguaje natural y la traducción. El tamaño de los pesos de ambos modelos rondan los 560MB cada uno y poseen configurados alrededor de 148 millones de parámetros. Fueron elegidos por sobre el segundo modelo debido a que poseían mayor fidelidad en la traducción y facilidad en su utilización.

El segundo modelo evaluado fue el *facebook/mbart-large-50-many-to-many-mmt* construido por Meta con la finalidad de trabajar en el enmascaramiento de fragmentos y la permutación de oraciones. Posee la capacidad de traducir directamente entre cualquier par de idiomas sin utilizar el inglés como puente. También está construido en arquitectura Transformer, el modelo posee configurado alrededor de 610 millones de parámetros, siendo el tamaño de los pesos aproximadamente los 2.4GB. A diferencia del par de modelos seleccionados, al utilizar éste modelo con **pipeline** comenzaba a fallar en la traducción, debiendo recurrir a métodos auxiliares como tokenizadores para que se traduzca de manera correcta.

3. ¿Dónde se equivoca? Mostrá ejemplos reales de tu notebook y explicá **por qué creés** que se equivoca, relacionándolo con los datos con los que fue entrenado.

Los modelos *Helsinki-NLP/opus-mt* fueron los que mejor desempeño tuvieron, y por ello fueron elegidos. Se detectó en su uso que cuando se ingresa una frase en un idioma que no pertenece al par entrenado, no traduce correctamente:




Éste es un comportamiento esperable, ya que los modelos fueron entrenados en un solo par de idiomas. Es una restricción que hay que tener en cuenta para el usuario a la hora de construir la interfaz.

El modelo *facebook/mbart-large-50-many-to-many-mmt* comenzó a fallar en las traducciones que generaba, mezclando la frase traducida con la etiqueta de idioma:

```
Frase traducida: ES was very happy with the food the petshoper sold me My dogEN was very happy with the food the petshoper sold me.
Función pipeline: translation_EN_to_ES
Tipo de dato de la traducción: <class 'str'>
```

Para evitar éstos errores, se deben recurrir a métodos tokenizadores, dejando de lado el objeto **pipeline**:

```python
from transformers import MBartForConditionalGeneration, MBart50TokenizerFast

nombre_modelo = "facebook/mbart-large-50-many-to-many-mmt"

# Indicamos idiomas de origen y destino al tokenizador
tokenizador = MBart50TokenizerFast.from_pretrained(
    nombre_modelo,
    src_lang="en_XX",   # inglés
    tgt_lang="es_XX"    # español
)
modelo = MBartForConditionalGeneration.from_pretrained(nombre_modelo)

texto = "My dog was very happy with the food the pet shop sold me."

inputs = tokenizador(texto, return_tensors="pt")

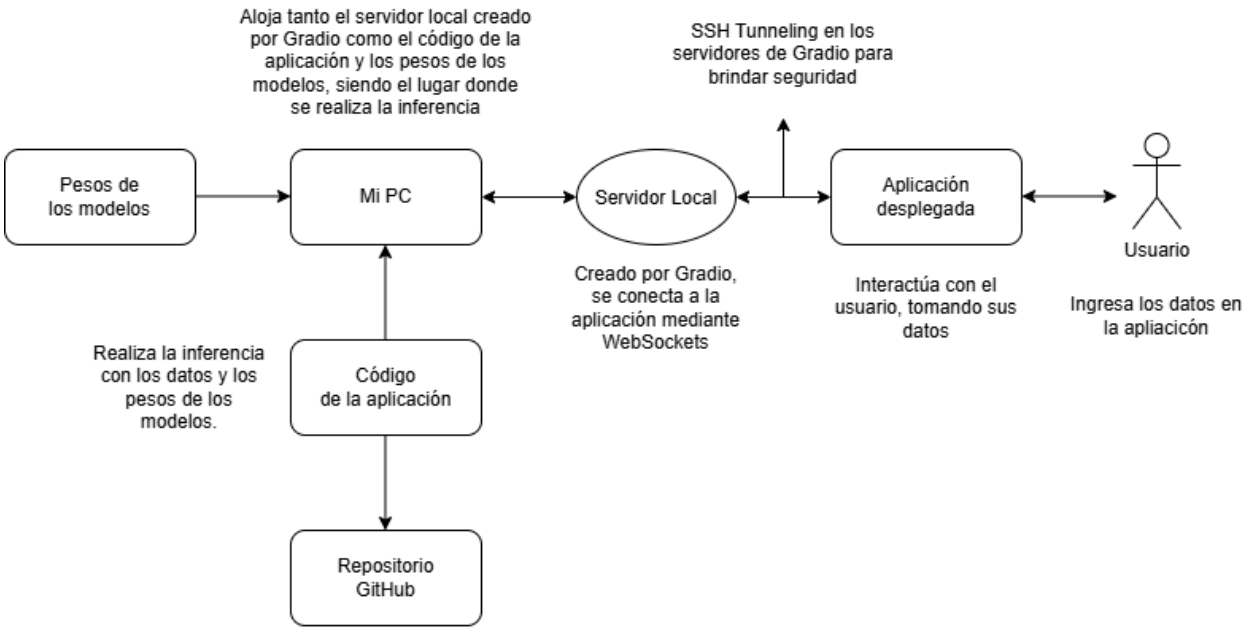
# Definimos la generación arranque con el token de español
output_ids = modelo.generate(
    **inputs,
    forced_bos_token_id=tokenizador.lang_code_to_id["es_XX"]
)

traduccion = tokenizador.batch_decode(output_ids, skip_special_tokens=True)[0]
print(traduccion)
```

Ésta solución no es errónea, pero nos aleja de la consigna de utilizar el objeto **pipeline**.

**C. Dónde corre cada cosa**

5. ¿Dónde vive el código y dónde vive el modelo?
Hacé un esquema simple (cajas y flechas, a mano o en cualquier herramienta) con los lugares que intervienen en tu app: tu compu, GitHub, la plataforma de deploy y Hugging Face. En cada caja anotá qué hay ahí, y marcá con flechas qué viaja de un lugar a otro cuando alguien usa la app. Tiene que quedar claro: ¿dónde está guardado tu código? ¿Dónde se ejecuta? ¿Dónde están los pesos del modelo y dónde se hace la inferencia?




6. ¿Usaste `pipeline` o `InferenceClient` en la app publicada? Justificalo con números (memoria del plan gratuito, peso del modelo, tiempo de respuesta). ¿Qué pasaría si hubieras elegido la otra?

Me decanté por el uso de **pipeline**, ya que el cupo limitado provisto por el plan gratuito de Hugging Face me dificulta la aplicación de **InferenceClient.**

**D. El deploy**

7. ¿Dónde deployaste y por qué ahí? Compará tu elección con **al menos otras dos** opciones vistas en clase (hosting compartido, VPS, PaaS, serverless, nubes elásticas) y explica por qué las descartaste. ¿Por qué tu app necesita un servidor que quede encendido?

Realicé el deploy en un servidor local creado por Gradio. Mis razones para tal decisión fueron las siguientes:

1. Frente a **PaaS**: Render, por ejemplo, ofrece un límite de RAM gratuito de 512MB, el cual es limitado ya que los dos modelos que componen la aplicación pesan aproximadamente 560MB (cada uno por sí solo). La única forma es utilizar un modelo a la vez, apagando la aplicación entre idiomas, y con la demora que ocasiona la carga del modelo seleccionado.

2. Frente a **hosting compartido**: los hostings compartidos están pensados para PHP, invalidando instalar **torch**, **transformers** o cualquier otra librería de Python, en clara desventaja contra la PC. Además, un hosting compartido accesible brinda aproximadamente 1GB de RAM y CPU, recursos que poseo por demás en mi PC.

**E. Lo que se rompió**

8. Documenta al menos dos errores reales que te aparecieron durante el trabajo. Para cada uno:
   - El **mensaje de error** completo (captura o texto copiado);
   - **Dónde apareció**: en tu compu, en la plataforma de deploy o en Hugging Face;
   - **Qué lo causaba**
   - **Cómo lo resolviste**, paso a paso, como para que otra persona pueda seguirlo.

1er Error: Funcionalidad **traductor** obsoleta:
A partir de la versión 5.0 de la librería Transformers, la tarea *translator* es eliminada, siendo reemplazada por modelos LLM, observando que se obtienen mejores resultados en la traducción con éstos últimos. Utilizarlos a la larga iba a complicar el desarrollo de la aplicación ya que los modelos LLM son considerablemente más pesados que los de traducción. El error que aparece al tratar de utilizar la tarea *traslator* es similar al siguiente:

```
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_611/4635897.py in <cell line: 0>()
----> 1 pipe_traduccion = pipeline("translation", model="Helsinki-NLP/opus-mt-es-en")
      2 print(pipe_traduccion.task)
      3 resultado = pipe_traduccion('Estoy construyendo mi primer modelo de traducción. Estoy muy emocinado!')
      4 print(resultado)

 2 frames
/usr/local/lib/python3.13/dist-packages/transformers/pipelines/base.py in check_task(self, task)
   1362             return task, targeted_task, None
   1363
-> 1364         raise KeyError(f"Unknown task {task}, available tasks are {self.get_supported_tasks()}")
   1365
   1366     def register_pipeline(

KeyError: "Unknown task translation, available tasks are ['any-to-any', 'audio-classification', 'automatic-speech-recognition', 'depth-estimation', 'document-question-answering', 'feature-extraction', 'fill-mask', 'image-classification', 'image-feature-extraction', 'image-segmentation', 'image-text-to-text', 'keypoint-matching', 'mask-generation', 'ner', 'object-detection', 'sentiment-analysis', 'table-question-answering', 'text-classification', 'text-generation', 'text-to-audio', 'text-to-speech', 'token-classification', 'video-classification', 'zero-shot-audio-classification', 'zero-shot-classification', 'zero-shot-image-classification', 'zero-shot-object-detection']"
```

```
KeyError: 'The task translation has been removed from transformers. Use the `text-generation` pipeline with a chat model instead.
```

La solución a éste inconveniente es la instalación de versiones anteriores a la 5.0 de la librería Transformers.

```
%pip install "transformers<5" torch sentencepiece sacremoses
```

2. No todos los modelos de traducción funcionan de manera óptima con **pipeline**. Modelos de traducción directa (como los **opus**) tienen buen desempeño, pero al utilizar modelos multilenguaje el resultado de la traducción presentaba alteraciones en el orden de las palabras:

```
Frase traducida: ES was very happy with the food the petshoper sold me My dogEN was very happy with the food the petshoper sold me.
Función pipeline: translation_EN_to_ES
Tipo de dato de la traducción: <class 'str'>
```

Para solucionar éste inconveniente, se deben definir los *tokens* de idioma de origen y destino con métodos tales como *MBartForConditionalGeneration* y *MBart50TokenizerFast*:

```python
from transformers import MBartForConditionalGeneration, MBart50TokenizerFast

nombre_modelo = "facebook/mbart-large-50-many-to-many-mmt"

# Indicamos idiomas de origen y destino al tokenizador
tokenizador = MBart50TokenizerFast.from_pretrained(
    nombre_modelo,
    src_lang="en_XX",   # inglés
    tgt_lang="es_XX"    # español
)
modelo = MBartForConditionalGeneration.from_pretrained(nombre_modelo)

texto = "My dog was very happy with the food the pet shop sold me."

inputs = tokenizador(texto, return_tensors="pt")

# Definimos la generación arranque con el token de español
output_ids = modelo.generate(
    **inputs,
    forced_bos_token_id=tokenizador.lang_code_to_id["es_XX"]
)

traduccion = tokenizador.batch_decode(output_ids, skip_special_tokens=True)[0]
print(traduccion)
```

9. Límites de tu solución:

¿Qué pasa si la usan 1000 personas a la vez?
En un principio ésta app está diseñada para pocos usuarios, pero el desempeño no sería el ideal ya que el servidor de la aplicación sería mi propia computadora, con las limitaciones que eso conlleva: la recepción de las peticiones, almacenamiento, la cantidad de inferencias al mismo tiempo, etc.

¿Si Hugging Face cambia las condiciones del plan gratuito (como ya pasó con los Spaces)?
Si los cambios son para mejor, evaluaría migrar la aplicación a Hugging Face. Actualmente los beneficios gratuitos de Hugging Face son limitados.

¿Qué cambiarías para que sea un producto real?
Evaluar opciones pagas para poder expandir el almacenamiento y así migrar mi aplicación a servidores externos.

**F. Vocabulario**

10. Tomá **5 líneas de tu `app.py`** y señalá en ellas: una clase, una instancia, un método, un atributo, un parámetro y su argumento, y un valor de retorno.






Link a la aplicación: …

Link a repositorio de GitHub: https://github.com/sebsmedina/seminario_2026_proyecto_integrador/tree/main/proyecto_integrador

Aclaración: para la construcción de la aplicación se utilizó Dola AI como asistente de IA para resolución de los errores, decisiones técnicas y construcción de README.md de la aplicación.